

CENTRO UNIVERSITÁRIO DE MACEIÓ - UNIMA Afya
CURSO DE CIÊNCIA DA COMPUTAÇÃO

Maria Layanne Rodrigues Da Silva

Avaliação Prática PBL: Componentes de Compiladore

Maceió
2026

1. Introdução

Os compiladores são ferramentas fundamentais no desenvolvimento de software, sendo responsáveis por traduzir programas escritos em linguagens de alto nível para uma linguagem compreensível pelo computador. Durante esse processo, diversas estruturas e mecanismos são utilizados para armazenar e organizar informações necessárias à análise e validação do código-fonte.

Entre esses mecanismos destaca-se a Tabela de Símbolos, estrutura responsável por registrar informações sobre identificadores, como nomes de variáveis e seus respectivos tipos, permitindo que o compilador realize verificações semânticas e controle adequadamente os diferentes escopos de um programa.

O presente trabalho tem como objetivo implementar um Gerenciador de Tabela de Símbolos utilizando uma pilha de tabelas hash, simulando o comportamento empregado por compiladores durante o gerenciamento de escopos aninhados. A aplicação permite criar e remover escopos, declarar variáveis e realizar buscas respeitando as regras de visibilidade definidas pelos diferentes níveis de escopo.

A implementação foi desenvolvida na linguagem Python utilizando estruturas de dados nativas, possibilitando a demonstração prática dos conceitos estudados na disciplina de Compiladores. Este projeto também compõe o processo avaliativo da disciplina, funcionando como uma atividade prática para consolidar os conhecimentos abordados em sala de aula.

2. Metodologia de Implementação

A implementação foi desenvolvida utilizando a linguagem Python e fundamenta-se no conceito de pilha de tabelas hash para representar os diferentes escopos de um programa.

A estrutura principal do sistema consiste em uma lista Python (`list`), utilizada como uma pilha de escopos. Cada escopo é representado por um dicionário (`dict`), estrutura baseada em tabela hash responsável por armazenar pares contendo o nome da variável e seu respectivo tipo.

Inicialmente, o sistema cria automaticamente um escopo global, que permanece ativo durante toda a execução do programa. Novos escopos podem ser adicionados sobre esse escopo global, formando uma pilha de escopos aninhados. Sempre que um novo escopo é criado, uma nova tabela hash é adicionada ao topo da pilha. Da mesma forma, quando um escopo é removido, a tabela hash correspondente é retirada da estrutura por meio da operação de desempilhamento.

A funcionalidade de declaração de variáveis foi implementada por meio do método `declarar(variavel, tipo)`, responsável por inserir novas entradas na tabela hash do escopo atual. Antes da inserção, o sistema verifica se a variável já foi declarada naquele mesmo escopo, evitando duplicidades.

A busca de variáveis é realizada pelo método `buscar(variavel)`, que percorre os escopos do mais interno para o mais externo. Esse comportamento reproduz o mecanismo utilizado pelos compiladores para localizar identificadores e respeitar as regras de visibilidade entre escopos aninhados.

Além das funcionalidades principais, foram implementadas rotinas de validação para nomes de variáveis e tipos permitidos, bem como um menu interativo em terminal para facilitar a execução e os testes da aplicação.

Dessa forma, a solução proposta demonstra na prática a utilização conjunta de pilhas e tabelas hash para o gerenciamento de símbolos, reproduzindo um dos componentes fundamentais presentes na fase de análise semântica de compiladores.

3. Casos de Teste

Os casos de teste foram realizados por meio da execução do programa no ambiente Google Colab. As evidências apresentadas nesta seção consistem em capturas de tela (prints) obtidas durante a execução do sistema, demonstrando as entradas fornecidas pelo usuário e as respectivas saídas produzidas pela aplicação. Esses registros foram utilizados para validar o funcionamento das funcionalidades implementadas, incluindo a criação e remoção de escopos, declaração de variáveis, busca de símbolos e visualização da tabela de símbolos.

Caso de Teste 1 – Criação de Escopo e Declaração de Variável

Entrada

1 - Criar escopo

3 - Declarar variável

Nome da variável: idade

Tipo da variável: int

Saída Esperada

Novo escopo criado.

Declarada: idade -> int

Objetivo

Verificar se o sistema cria corretamente um novo escopo e registra uma variável na tabela de símbolos.

```
=== MENU ===  
... 1 - Adicionar escopo  
      2 - Remover do escopo  
      3 - Declarar variável  
      4 - Buscar variável  
      5 - Mostrar tabela  
      0 - Sair  
Escolha: 1  
Novo escopo criado.
```

```
=== MENU ===  
1 - Adicionar escopo  
2 - Remover do escopo  
3 - Declarar variável  
4 - Buscar variável  
5 - Mostrar tabela  
0 - Sair  
Escolha: 3
```

Exemplos de nomes válidos:

```
x  
idade  
nome_cliente  
_total  
valor123
```

Nome da variável: idade

Tipos disponíveis:

```
int  
float  
string  
char  
bool
```

Tipo da variável: int
Declarada: idade -> int

Caso de Teste 2 – Busca de Variável Existente

Entrada

4 - Buscar variável

Variável: idade

Saída Esperada

Tipo encontrado: int

Objetivo

Verificar se a busca encontra corretamente uma variável previamente declarada.

Tipo da variável: int
Declarada: idade -> int

```
=== MENU ===  
1 - Adicionar escopo  
2 - Remover do escopo  
3 - Declarar variável  
4 - Buscar variável  
5 - Mostrar tabela  
0 - Sair  
Escolha: 4  
Variável a buscar: idade  
Tipo encontrado: int
```

Caso de Teste 3 – Busca de Variável Inexistente

Entrada

4 - Buscar variável

Variável: salario

Saída Esperada

Variável não encontrada.

Objetivo

Verificar o comportamento do sistema quando a variável não existe em nenhum escopo.

```
=== MENU ===  
1 - Adicionar escopo  
2 - Remover do escopo  
3 - Declarar variável  
4 - Buscar variável  
5 - Mostrar tabela  
0 - Sair  
Escolha: 4  
Variável a buscar: salario  
Variável não encontrada.
```

Caso de Teste 4 – Escopos Aninhados

Entrada

- 1 - Criar escopo
- 3 - Declarar variável

Nome da variável: x

Tipo da variável: int

- 1 - Criar escopo
- 3 - Declarar variável

Nome da variável: x

Tipo da variável: float

- 4 - Buscar variável

Variável: x

Saída Esperada

Novo escopo criado.

Declarada: x -> int

Novo escopo criado.

Declarada: x -> float

Tipo encontrado: float

Objetivo

Verificar se a variável declarada no escopo mais interno possui prioridade durante a busca.

1º

```
=== MENU ===
... 1 - Adicionar escopo
      2 - Remover do escopo
      3 - Declarar variável
      4 - Buscar variável
      5 - Mostrar tabela
      0 - Sair
Escolha: 1
Novo escopo criado.
```

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3
```

Nome da variável: x

Tipo da variável: int
Declarada: x -> int

2º

```
=== MENU ===
... 1 - Adicionar escopo
      2 - Remover do escopo
      3 - Declarar variável
      4 - Buscar variável
      5 - Mostrar tabela
      0 - Sair
Escolha: 1
Novo escopo criado.
```

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3
Nome da variável: x
```

Tipo da variável: float
Declarada: x -> float

3º

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 4
Variável a buscar: x
Tipo encontrado: float
```

Caso de Teste 5 – Remoção de Escopo

Entrada

2 - Remover escopo

4 - Buscar variável

Variável: x

Saída Esperada

Escopo removido.

Tipo encontrado: int

Objetivo

Verificar se a remoção do escopo restaura a visibilidade da variável declarada no escopo externo.

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 2
Escopo removido.
```

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 4
Variável a buscar: x
Tipo encontrado: int
```


Caso de Teste 6 – Redeclaração de Variável

Entrada

3 - Declarar variável

Nome da variável: idade

Tipo da variável: int

3 - Declarar variável

Nome da variável: idade

Tipo da variável: float

Saída Esperada

Declarada: idade -> int

Erro: 'idade' já declarada neste escopo.

Objetivo

Verificar se o sistema impede a declaração duplicada de variáveis dentro do mesmo escopo.

```
=== MENU ===
1 - Adicionar escopo
... 2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3

Nome da variável: idade

Tipo da variável: int
Declarada: idade -> int

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 3

Nome da variável: idade

Tipo da variável: float
Erro: 'idade' já declarada neste escopo.
```

Caso de Teste 7 – Proteção do Escopo Global

Objetivo

Verificar se o sistema impede a remoção do escopo global, garantindo que sempre exista pelo menos um escopo ativo na tabela de símbolos.

Entrada

2 - Remover escopo

Saída Esperada

Erro: o escopo global não pode ser removido.

Evidência

A execução demonstra que, ao tentar remover o único escopo existente (escopo global), o sistema bloqueia a operação e informa o usuário por meio de uma mensagem de erro apropriada.

Esse comportamento garante a integridade da estrutura de gerenciamento de escopos, uma vez que todo programa deve possuir ao menos um escopo ativo para armazenar símbolos.

```
Nome da variável: idade
***
Tipos disponíveis:
int
float
string
char
bool

Tipo da variável: float
Declarada: idade -> float

=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 2
Erro: Não é possível remover o Escopo Global.
```

Caso de Teste 8 – Visualização da Tabela de Símbolos

Entrada

5 - Mostrar tabela

Saída Esperada

Escopos atuais:

Escopo 0: { }

Escopo 1: {'x': 'int', 'nome': 'string'}

Escopo 12: {'x': 'float', 'idade': 'int'}

Objetivo

Verificar se o sistema exibe corretamente a estrutura da pilha de escopos e suas respectivas tabelas hash.

```
=== MENU ===
1 - Adicionar escopo
2 - Remover do escopo
3 - Declarar variável
4 - Buscar variável
5 - Mostrar tabela
0 - Sair
Escolha: 5

Escopos atuais:
Escopo 0: {}
Escopo 1: {'x': 'int', 'nome': 'string'}
Escopo 2: {'x': 'float', 'idade': 'int'}
```